Topological Data Analysis for Multi-Asset Crypto Mean-Reversion Trading

Table of Contents

1. Introduction
    - 1.1 What is Topological Data Analysis?
    - 1.2 Persistent Homology
    - 1.3 From Time Series to Point Clouds
    - 1.4 Persistent Homology
    - 1.5 Persistence Landscapes
2. Building a Liquid Crypto Basket
    - 2.1 Selection Criteria
    - 2.2 Recommended Basket (40+ assets)
3. Extracting Clusters with TDA
    - 3.1 Data Preparation
    - 3.2 Computing the Distance Matrix
    - 3.3 Running Vietoris-Rips and Obtaining Persistence Diagrams
    - 3.4 Identifying Persistent H_0 Features
    - 3.5 From Birth Scales to Actual Asset Groups
4. Trading Strategy: Cluster Mean-Reversion
    - 4.1 Core Idea
    - 4.2 Entry Rules
    - 4.3 Position Sizing
    - 4.4 Exit Rules
    - 4.5 Risk Management
5. Complete Code Example

    - 5.1 Environment Setup
    - 5.2 Fetching Data
    - 5.3 TDA Pipeline
    - 5.4 Signal Generation and Backtesting Skeleton
6. Practical Considerations
    - 6.1 Parameter Selection
    - 6.3 Data Sources

Conclusion

1. Introduction

Classical pair trading is limited to two assets and assumes stable linear relationships. In cryptocurrency markets, where assets move in complex, non-linear patterns, we need a more powerful tool. **Topological Data Analysis (TDA)** offers a way to capture the **shape** of market data and detect persistent groups of assets that move together. This tutorial will guide you through building a multi-asset mean-reversion strategy using TDA on a large basket of liquid cryptocurrencies.

1.1 What is Topological Data Analysis?

Topology studies shapes and their connectivity. TDA applies these ideas to data: we treat a set of data points as a **point cloud** , where its number of dimensions = number of assets and the number of points = number of days in the sliding window.

The following questions can be asked abot the cloud:

- Which points are close enough to form a cluster?
- Do the points form loops or other structures?

For trading, we are mostly interested in **clusters** – groups of assets that move together persistently.

1.2 Homology

Homology is a mathematical tool that counts holes in a shape. If you have a shape, homology gives you numbers that answer the question: "How many holes of different dimensions does this shape have?"

- H_0 answers: "How many separate pieces (**clusters**) are there?" Here clusters → groups of assets that stick together. In the context of trading:

  A **persistent** cluster (one that exists over a range of radii) means those assets consistently move together (co-movement). For example, if BTC, ETH, and SOL always form a cluster that doesn't break apart until a very large radius, they have a strong co-movement relationship. Hence these three form a **mean-reversion group**: when one of them drifts away from the others, it is likely to come back.

- H_1 answers: "How many 1-dimensional holes (loops) are there?"
- H_2 answers: "How many 2-dimensional holes (voids/cavities) are there?" (like the inside of a soccer ball)

The "H" is used because it's the standard notation in algebraic topology. The groups that count these holes are also named as *homology groups*, and by convention they are labeled H_0, H_1, H_2, ...

The subscript number (0, 1, 2, ...) refers to the dimension of the hole being counted.

H_0 – Zero-Dimensional Holes:

In topology, a 0-dimensional hole is actually a gap between connected components.

In your crypto data: H_0 counts how many distinct clusters of assets exist at a given scale. For example, if $H_0 = 3$, you have three separate groups of cryptocurrencies that move together.

H_1 – One-Dimensional Holes:

These are the holes you normally think of, like the hole in a donut or the loop in a circle.

- In the context of crypto related data: H_1 counts how many loops or cycles exist in the relationship network. For example, if $H_1 = 1$, there's one circular relationship among your assets (like A→B→C→A). In the context of trading, cycles are mostly theoretical or limited to very specific, high-frequency scenarios. Even classical **triangular arbitrage** (the simplest H_1 case) is very difficult to profit from in crypto due to:
  - Exchange fees
  - Slippage

- Execution speed requirements
- Thin order books on some pairs

An example of a triangular arbitrage trade: BTC → ETH → SOL → BTC

Relationship between Homology groups and Betti numbers: Homology groups (H_0, H_1, H_2, ...) are the actual "containers" that store information about holes (features). Betti numbers (β_0, β_1, β_2, ...) are simply the counts of how many holes each container holds.

1.3 From Time Series to Point Clouds

Suppose we have *n* cryptocurrencies and we look at their daily log-returns over a window of *w* days. Each day gives us a point in *n* -dimensional space (the returns of all assets that day). Over *w* days we obtain *w* points, which represent our point cloud.

1.4 Persistent Homology

Persistent homology tracks how clusters appear and disappear as we vary a **scale parameter** ε (the distance threshold for connecting points).

- **Birth** of a cluster: at some ε, a set of points becomes connected (they form a cluster). To extract members of a specific cluster shown in the persistence diagram, use its own birth scale.
- **Death** of a cluster: at a larger ε, that cluster merges with another one. Death scale of a cluster = Birth scale of the next larger cluster. If you want the cluster that results from a merger, look at the birth scale of that merged cluster (which is the death scale of the merging ones).
- **Persistence** = death − birth: how long the cluster lived on its own. Long-lived (persistent) clusters are considered **real** structures, not noise.

The collection of birth/death pairs can be nicely visualized in a **persistence diagram**, which is a plot where each hole (feature) is a dot:

- The x-coordinate is its birth time (scale ε when a feature appears).
- The y-coordinate is its death time (scale ε when it disappears).

- A dot far from the diagonal (where birth=death) represents a persistent feature that lived for a long time. These are considered the "real" topological signals. Dots near the diagonal are short-lived and are considered "noise."

The diagram itself is a visual representation of the homology groups across all scales.

- All points with **dimension 0** belong to H_0 information.
- All points with **dimension 1** belong to H_1 information.

The **Betti number** at a specific scale ε is simply the number of points in the diagram that are **alive** at that scale:

- The scale ε is fully represented, i.e. it's the numerical values on both axes.
- For any chosen ε, you can determine which features are alive by checking each point's birth and death against ε.
- The diagram is a static snapshot that encodes the entire evolution of features across all ε.

For clusters we only look at dimension 0 (H_0). For a nice description of persistence diagrams refer to the online article: Understanding the shape of data (II) | Quantdare

1.5 Persistence Landscapes

To turn diagrams into numbers the paper "*Topological Machine Learning for Financial Crisis Detection: Early Warning Signals from Persistent Homology*" by the authors Guritanu et al. computes **landscapes** and their **L^2 norm,** which is a single number that measures the overall "topological activity" in each sliding window. That number is used as an early warning signal for crises.

- A **Persistence Landscape** turns the information from the persistence diagram into a series of functions.
- Each "layer" of the landscape (k=1, 2, 3…) corresponds to the most prominent features.
    - **Layer 1 (k=1)** captures the single most persistent feature.
    - **Layer 2 (k=2)** captures the second most persistent, and so on.
- The height of the "mountain" at any point shows how significant a topological feature is at that scale.

For your trading strategy, you are not trying to create a single crisis indicator. Instead, you want to:

1. **Identify persistent clusters (H_0)** from the persistence diagram.
2. **Extract the actual assets** that belong to each cluster.
3. **Trade mean-reversion** within those clusters.

2. Building a Liquid Crypto Basket

2.1 Selection Criteria

We need a basket large enough to contain meaningful clusters, but every asset must be liquid enough to trade without excessive slippage. Minimum requirements:

- **24h trading volume** > $50 million (on major exchanges)
- **Listed on at least 10 exchanges** (to ensure you can enter/exit)
- **Market cap** in top 100

2.2 Recommended Basket (40+ assets)

Here is a suggested basket as of 2026. **Always verify current liquidity before trading.**

Tier	Tickers
Core	BTC, ETH, BNB, SOL, XRP, ADA, DOGE, DOT, AVAX, MATIC
High-liquid altcoins	LINK, UNI, LTC, BCH, XLM, MONERO, EOS, TRX, VET, FIL
Layer-2 & DeFi	ARB, OP, CRV, AAVE, MKR, SNX, GRT
Emerging	APT, SUI, SEI, TIA, INJ, RNDR, FET

Total: **~35–40 assets**. This size is large enough to find clusters but still computationally manageable.

3. Extracting Clusters with TDA

Our goal is to find **groups of cryptocurrencies that move together persistently** (clusters). For this, we need to use a mathematical object called a **persistence diagram** that comes from Topological Data Analysis (TDA). This diagram tells us about the "birth" and "death" of clusters as we look at different distance scales.

But the function as written is **incomplete** – it only gives you the "birth" and "death" times of clusters, not which assets are actually in those clusters. You need extra steps to get the actual asset names.

We will use Python with libraries: giotto-tda (giotto-ai/giotto-tda: A high-performance topological machine learning toolbox in Python), numpy, pandas, yfinance (or ccxt for crypto), and scipy. In the following Python code, instead of the Python Package cctx, you could use the Python code written as part of Data Engineering project for acquiring crypto data from Binance.

3.1 Data Preparation

```
import numpy as np
import pandas as pd
import yfinance as yf  # for stocks; use ccxt for crypto
from giotto.homology import VietorisRipsPersistence
from sklearn.preprocessing import StandardScaler

# Download daily close prices
tickers = ['BTC-USD', 'ETH-USD', 'BNB-USD', ...]  # your basket
data = yf.download(tickers, start='2020-01-01', end='2025-01-01')['Adj Close']
returns = data.pct_change().dropna()

# Create sliding windows of 30 days each
window_size = 30
windows = []
for i in range(len(returns) - window_size + 1):
    window = returns.iloc[i:i+window_size]
    scaler = StandardScaler()
    scaled = scaler.fit_transform(window)  # shape (window_size, n_assets)
    windows.append(scaled)
point_clouds = np.array(windows)  # shape (n_windows, window_size, n_assets)
```

3.2 Computing the Distance Matrix

For each window, we need the **pairwise distance** between assets. As the distance metric we will use correlation (ρ). To this end, we need to transform it because correlation itself measures **similarity**, not distance. In topological data analysis (and many other applications), correlation is often converted into a distance so that it satisfies the properties required for a metric: non-negativity, symmetry, and the triangle inequality.

The most common and mathematically sound choice for financial time series is given by

$$d = \sqrt{2(1-\rho)}$$

This yields a proper distance matrix that can be plugged directly into the Vietoris-Rips filtration, allowing you to detect persistent clusters and cycles based on correlation patterns among assets.

In Python code we get:

```
def correlation_distance(returns_window):
    corr = np.corrcoef(returns_window.T)
    dist = np.sqrt(2 * (1 - corr))
    return dist
```

3.3 Running Vietoris-Rips and Obtaining Persistence Diagrams

We use **giotto-tda** (giotto-ai/giotto-tda: A high-performance topological machine learning toolbox in Python) to compute persistence diagrams for each window.

```
vr = VietorisRipsPersistence(
    metric='precomputed',           # we supply distance matrices
    homology_dimensions=[0],        # only need H₀
    max_edge_length=0.5,
    n_jobs=-1
)

# Precompute distance matrices for all windows
dist_mats = np.array([correlation_distance(w) for w in point_clouds])

# Fit transform – returns list of diagrams (one per window)
diagrams = vr.fit_transform(dist_mats)
```

where `diagrams[i]` is an array of shape (n_features, 3) where columns are [dimension, birth, death]. We only keep rows where `dimension == 0`.

3.4 Identifying Persistent H_0 Features

We loop through each window's diagram and collect features with high persistence.

```
min_persistence = 0.1   # threshold, adjust based on your data
persistent_clusters_per_window = []

for window_idx, dgm in enumerate(diagrams):
    h0 = dgm[dgm[:, 0] == 0]          # filter dimension 0
    features = []
    for feat in h0:
        persistence = feat[2] - feat[1]
        if persistence > min_persistence:
            features.append({
                'birth': feat[1],
                'death': feat[2],
                'persistence': persistence
            })
    persistent_clusters_per_window.append(features)
```

Now `persistent_clusters_per_window[window_idx]` is a list of persistent H_0 features detected in that window.

3.5 From Birth Scales to Actual Asset Groups

For each persistent feature, we need the exact set of assets. We use the **birth scale** to cut the hierarchical clustering tree.

```
from scipy.cluster.hierarchy import linkage, fcluster
from scipy.spatial.distance import squareform


def get_cluster_assets_at_scale(dist_matrix, birth_scale, tickers):
    # Perform hierarchical clustering using single linkage
    Z = linkage(squareform(dist_matrix), method='single')
    # Cut the tree at height = birth_scale
    labels = fcluster(Z, t=birth_scale, criterion='distance')
    # Group tickers by label
    groups = {}
    for idx, label in enumerate(labels):
        groups.setdefault(label, []).append(tickers[idx])
    # Which group corresponds to our feature?
    # For simplicity, we can take the largest group that appears at this scale.
    # In practice, you may need a more robust mapping.
    largest = max(groups.values(), key=len)
    return largest
```

Important: A single birth scale can produce multiple clusters. The persistent feature in the diagram corresponds to **one** of them. A heuristic is to take the **largest** group that exists at that scale, because the persistent feature is likely a major component. Alternatively, you can use a dedicated persistence-based clustering algorithm like **ToMATo**.

For a production system, you might store the feature's **birth** and later, when you want to trade, you re-run the clustering at that birth scale and use the resulting groups. The birth scale can be used as a parameter for a clustering algorithm like **DBSCAN** with `eps = birth_scale`.

4. Trading Strategy: Cluster Mean-Reversion

4.1 Core Idea

Within each persistent cluster, assets tend to move together. When one asset deviates significantly from the cluster's average, we expect it to revert. We therefore go **long** the most underperforming asset and **short** the most overperforming asset in the same cluster.

4.2 Entry Rules

For a given cluster with assets $C = [a_1, a_2, \ldots, a_k]$ observed at time t (the most recent window):

1. Compute the **cluster mean return** $\mu = \text{mean}(r_i)$ for all i in C.
2. Compute the **standard deviation** σ of returns within the cluster.
3. For each asset, compute its **z-score**: $z_i = (r_i - \mu) / \sigma$.
4. Identify the asset with the **lowest** z-score (most underperforming) and the asset with the **highest** z-score (most overperforming).
5. If the spread z_max – z_min > threshold (e.g., 2.5), enter a trade:
    - **Long** the underperformer
    - **Short** the overperformer
    - Use equal notional amounts (dollar neutral within the cluster).

4.3 Position Sizing

Allocate a fixed fraction of capital per cluster (e.g., 2%). If the spread is exceptionally large, you can increase the size (e.g., 1.5× for spread > 3.5). Always ensure that total exposure across all clusters does not exceed a limit (e.g., 30% of capital).

4.4 Exit Rules

General idea: Exit when relationships normalize or the cluster structure breaks.

- **Profit target:** Close when the spread narrows to, say, 0.5 (or when the z-scores cross zero).
- **Stop loss:** Close if the spread widens by 50% (e.g., from 3.0 to 4.5).
- **Time stop:** Close after a fixed number of days (e.g., 10).
- **Cluster break:** If the cluster dissolves (i.e., it no longer appears in the most recent persistence diagram), exit **immediately**.

4.5 Risk Management

- **Beta-neutrality:** Within the cluster, the long/short should be naturally hedged against market moves because both assets belong to the same persistent group.
- **Correlation with other clusters:** Monitor that your trades are not all concentrated in highly correlated clusters.
- **Regime filter:** In high-volatility environments (e.g., BTC daily moves > 5%), reduce position sizes by half.

5. Complete Code Example

Here is a simplified but working skeleton that puts everything together. (Note: you need to install giotto-tda, ccxt, etc.)

```
import numpy as np
import pandas as pd
import ccxt
from giotto.homology import VietorisRipsPersistence
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import linkage, fcluster
from scipy.spatial.distance import squareform
import time

# ======================
```

```
# 1. Fetch data
# ======================
exchange = ccxt.binance()
tickers = ['BTC/USDT', 'ETH/USDT', 'BNB/USDT', 'SOL/USDT', 'XRP/USDT']  # expand to 40+

def fetch_returns(tickers, days=500):
    dfs = []
    for symbol in tickers:
        ohlcv = exchange.fetch_ohlcv(symbol, '1d', limit=days)
        df = pd.DataFrame(ohlcv,
columns=['timestamp','open','high','low','close','volume'])
        df['date'] = pd.to_datetime(df['timestamp'], unit='ms')
        df.set_index('date', inplace=True)
        df['returns'] = df['close'].pct_change()
        dfs.append(df['returns'])
    data = pd.concat(dfs, axis=1)
    data.columns = tickers
    data.dropna(inplace=True)
    return data

returns = fetch_returns(tickers, days=500)

# ======================
# 2. Sliding windows and distance matrices
# ======================
window_size = 30
windows = []
for i in range(len(returns) - window_size + 1):
    win = returns.iloc[i:i+window_size].values
    scaler = StandardScaler()
    win_scaled = scaler.fit_transform(win)
    windows.append(win_scaled)
windows = np.array(windows)  # shape (n_windows, window_size, n_assets)

def corr_dist(win):
    corr = np.corrcoef(win.T)
```

```
    return np.sqrt(2 * (1 - corr))

dist_mats = np.array([corr_dist(w) for w in windows])

# ======================
# 3. Compute persistence diagrams (only H0)
# ======================
vr = VietorisRipsPersistence(metric='precomputed', homology_dimensions=[0],
max_edge_length=0.5, n_jobs=-1)
diagrams = vr.fit_transform(dist_mats)

# ======================
# 4. Extract persistent clusters in the most recent window
# ======================
last_diagram = diagrams[-1]
h0_features = last_diagram[last_diagram[:, 0] == 0]
min_persist = 0.1
persistent = []
for feat in h0_features:
    pers = feat[2] - feat[1]
    if pers > min_persist:
        persistent.append(feat)

if len(persistent) == 0:
    print("No persistent clusters found.")
    exit()

# For simplicity, take the feature with largest persistence
main_feat = max(persistent, key=lambda f: f[2]-f[1])
birth = main_feat[1]

# Get distance matrix of the most recent window
last_dist = dist_mats[-1]

# Hierarchical clustering at birth scale
Z = linkage(squareform(last_dist), method='single')
```

```
labels = fcluster(Z, t=birth, criterion='distance')

# Group tickers by label
groups = {}
for idx, label in enumerate(labels):
    groups.setdefault(label, []).append(tickers[idx])

# Assume the persistent feature corresponds to the largest group
cluster_assets = max(groups.values(), key=len)
print("Trading cluster:", cluster_assets)

# ======================
# 5. Generate trading signal for the latest day
# ======================
latest_returns = returns.iloc[-1][cluster_assets]
mu = latest_returns.mean()
sigma = latest_returns.std()
z_scores = (latest_returns - mu) / sigma
sorted_assets = z_scores.sort_values()
long_asset = sorted_assets.index[0]
short_asset = sorted_assets.index[-1]
spread = z_scores[short_asset] - z_scores[long_asset]

if spread > 2.5:
    print(f"Signal: LONG {long_asset}, SHORT {short_asset}, spread = {spread:.2f}")
else:
    print("No trade, spread too small.")
```

This is a basic example. In a live trading system you would loop over each day, update windows, and manage open positions.

6. Practical Considerations

6.1 Parameter Selection

- **Window size**: 30 days is common, but you can experiment (20–60 days). Shorter windows react faster but may be noisier.
- **Persistence threshold**: Start with 0.1 (for correlation distance, which ranges 0–2). Plot histograms of persistence values to choose a meaningful cutoff.
- **Entry spread**: 2.5–3.0 works well in backtests. Adjust based on volatility.
- **max_edge_length** in Vietoris-Rips: set high enough to capture all relevant connections (e.g., 0.5 or 1.0).

6.2 Data Sources

- **Exchange data**: Use the Python code written as part of the Data Engineering project for acquiring historical crypto data from Binance.
- **On-chain data** (optional) can be used as additional confirmation, but not necessary for the core strategy.

Conclusion

This document serves as a roadmap to get up to speed with your bachelor thesis and help build a multi-asset mean-reversion strategy for cryptocurrencies using Topological Data Analysis. The key steps are:

1. Assemble a liquid basket of 40+ coins.
2. For each sliding window, compute persistence diagrams (H_0) using the Vietoris-Rips complex on correlation distances.
3. Identify persistent clusters (long-lived features).
4. Extract the actual asset lists by cutting the hierarchical tree at the feature's birth scale.
5. Within each cluster, go long the weakest and short the strongest when the z-score spread exceeds a threshold.
6. Manage risk with profit targets, stop losses, and cluster-break exits.

TDA gives you a dynamic, data-driven way to discover tradable groups, going far beyond static sector classifications. With the code and explanations provided, you can start experimenting and refining your own strategy.

References

Rudkin et al, On the topology of cryptocurrency markets, 2023 (PDF of this paper is available)

Guritanu et al., Topological Machine Learning for Financial Crisis Detection: Early Warning Signals from Persistent Homology